

SOLID 检查清单

AI 设计缺陷注入实验

第 44 组 · Phase 3 详细设计

2026 年 6 月 16 日

本清单记录 P3 阶段 AI 生成的初版类图/数据库设计中的 SOLID 违规项，以及团队对照实际代码落地后的修正与取舍。

对照基准：AI 原始设计（18 表、胖实体、JSON 快照、独立导出模块） vs 当前实现（`deploy/init.sql` + `backend/` 源码）。

一、S — 单一职责原则（Single Responsibility Principle）

检查问题：有没有类承担了过多职责？

AI 初稿结论：违反，共发现 3 处。

类名	涉及 职责 数	具体职责列表	违反说明	修正方案
<code>Task</code> (AI 初稿)	5	① 生命周期 ② 人员分配 ③ 标签配置 ④ 数据导出 ⑤ 状态判断	领域类承担多个变更原因	拆分为 <code>Task</code> （纯数据）+ 多个 <code>Service</code>
<code>Annotation</code> (AI 初稿)	3	① 数据实体 ② submit/overwrite ③ JSON 序列化	实体混入业务与转换逻辑	拆分为 <code>AnnotationPo</code> + <code>AnnotationService</code> + <code>AnnotationPersistenceService</code>
<code>User</code> (AI 初稿)	3	① 数据载体 ② login/logout ③ hasPermission	领域模型混入横切关注点	拆分为 <code>SysUser</code> + <code>AuthService</code> + <code>UserService</code>

实际实现对照：

类名	当前状态	说明
<code>Task</code> (PO)	已修正	仅承载字段，无业务方法
<code>AnnotationPo</code>	已修正	纯 POJO，持久化由 <code>AnnotationPersistenceService</code> 负责
<code>SysUser</code>	已修正	认证在 <code>AuthService</code> ，用户管理在 <code>UserService</code>

TaskService	部分违反	仍承担任务 CRUD、成员管理、文书管理、标注加载、导出占位等多项职责（约 340 行），是已知技术债
AnnotationPersistenceService	可接受	同时处理标注与裁定的级联持久化，但职责边界清晰（"标注数据读写"单一领域）

二、O — 开闭原则（Open/Closed Principle）

检查问题：新增需求类型是否需要修改现有代码？

AI 初稿结论：违反，共发现 4 处。

场景	AI 初稿是否违反	违反说明	修正方案
新增导出格式	是	<code>Task.exportData(format)</code> 需改 Task 类	模板方法模式 <code>AbstractExporter</code>
新增任务角色	是	<code>assignAnnotator/assignArbitrator</code> 硬编码	统一 <code>assignMember(roleType)</code>
标签配置历史追溯	是	<code>labelConfigSnapshot</code> JSON 死字段	提取 <code>LabelConfig</code> 独立实体
新增文件解析格式	轻度	可能退化为 if-else	<code>DocumentParserFactory</code>

实际实现对照：

场景	当前实现	评估
新增导出格式	导出改由前端 ZIP 实现，后端无导出模块	后端无需扩展；前端独立演进
新增任务角色	<code>task_member.role_in_task</code> 为 <code>ENUM('标注员','裁定者')</code> ，新增角色需改 <code>ENUM</code>	当前仅两种角色，够用；扩展需 migration
标签配置追溯	<code>annotation.guide_snapshot</code> 照	JSON 字段保存提交时快照 满足需求，未建独立 <code>label_config</code> 表
新增文书来源类型	<code>TaskDocumentFactory</code> 用 switch 分发 <code>GLOBAL/UPLOAD/RECREATE</code>	新增类型只需改 <code>Factory</code> ，不侵入 <code>TaskService</code> 主流程

新增文件格式	<code>DocumentTextExtractor</code> 用 switch 按扩展名解析	未用完整策略模式，但解析逻辑集中在一处
--------	--	---------------------

三、L — 里氏替换原则（Liskov Substitution Principle）

检查问题：子类是否可以替换父类使用？

AI 初稿结论：轻度违反 1 处。

类/继承关系	是否违反	违反说明	修正方案
<code>Label</code> 同时用于 L1/L2	轻度违反	<code>parentL1Id</code> 对 L1 无意义，调用方需先判断 level	工厂方法 <code>createL1/createL2</code> ，或拆表

实际实现对照：

设计	当前实现	评估
统一 Label 类	拆为 <code>LabelL1</code> / <code>LabelL2</code> 两个独立 PO 类，对应两张表	语义清晰，无替换歧义
用户角色继承	无继承层次， <code>SysUser.role</code> 字符串区分	避免了 AI 初稿中 Admin/Annotator 继承 User 的 LSP 问题

四、I — 接口隔离原则（Interface Segregation Principle）

检查问题：有没有接口太"胖"，包含了不需要的方法？

AI 初稿结论：违反 2 处。

接口/类	是否违反	违反说明	修正方案
Task 隐式接口	是	生命周期/分配/配置/导出方法横跨四领域	拆为 <code>ITaskLifecycle</code> / <code>IAssignable</code> / <code>IExportable</code>
<code>DocumentParser</code>	潜在风险	未来可能变胖	拆为 <code>IDocumentParser</code> / <code>IReasonExtractable</code>

实际实现对照：

设计	当前实现	评估
服务接口层	未定义 <code>ITaskLifecycle</code> 等 Java 接口， <code>Service</code> 为具体类	课程设计中的接口层未完全落地；Spring 可直接注入具体 <code>Service</code> ，对小团队可接受
Mapper 接口	MyBatis Mapper 按表拆分，粒度合理	每个 Mapper 只操作对应表
DocumentParser	无接口， <code>DocumentTextExtractor</code> 静态方法	简化为工具类，牺牲扩展性换取实现速度

五、D — 依赖倒转原则（Dependency Inversion Principle）

检查问题：高层模块是否直接依赖了低层模块的具体实现？

AI 初稿结论：违反 3 处。

高层模块	直接依赖	是否违反	修正方案
Task	<code>ExportLog</code> 具体类	是	引入 <code>IExportService</code> 接口
Annotation	JSON 序列化	是	<code>AnnotationDataConverter</code> 独立
Task	<code>labelConfigSnapshot</code> JSON	是	<code>LabelConfig</code> 独立实体 + <code>Repository</code>

实际实现对照：

模块	当前实现	评估
Service → 数据访问	所有 Service 依赖 Mapper 接口（MyBatis），由 Spring 注入	符合 DIP
标注数据存储	命题/关系存规范化表，非 JSON 反序列化	比 AI 方案更彻底地消除实体对 JSON 的依赖
指南快照	<code>guide_snapshot</code> 仍为 JSON，由 <code>AnnotationPersistenceService</code> 写入	可接受：快照为只读冗余，不影响核心查询路径
布局数据	<code>layout_json</code> 由 <code>GraphLayoutJsonCodec</code> 编解码	JSON 细节隔离在 Codec 中
导出	无 <code>ExportLog</code> 依赖	问题随导出方案调整而消除

六、SOLID 违规汇总表

SOLID 原则	检查问题	AI 初稿是否违反	实际实现状态	说明
S	Task 承担过多职责	违反	部分修正	PO 已纯化，但 TaskService 仍偏大
S	Annotation 混入业务+序列化	违反	已修正	AnnotationPo + AnnotationPersistenceService
S	User 混入认证+权限	违反	已修正	SysUser + AuthService + UserService
O	新增导出格式	违反	方案变更	前端导出，后端无此扩展点
O	新增任务角色	违反	ENUM 扩展	当前两种角色够用
O	配置历史追溯	违反	简化实现	guide_snapshot 替代独立表
O	文书来源/解析扩展	轻度	基本满足	Factory + Extractor 集中扩展
L	Label 语义双重性	轻度违反	已修正	LabelL1 / LabelL2 拆表拆类
I	Task 胖接口	违反	未建接口层	具体 Service 类，Mapper 已隔离
I	Parser 变胖风险	潜在	工具类替代	可接受的技术取舍
D	Task 依赖 ExportLog	违反	已消除	无后端导出模块
D	Annotation 依赖 JSON	违反	已修正	规范化表 + GraphLayoutJsonCodec
D	Task 依赖 JSON 配置	违反	简化	guide_snapshot 在 annotation 表

七、统计

统计项	AI 初稿	实际实现
SOLID 违规总数	13 项	3 项待改进（TaskService 职责、无 Service 接口层、ENUM 角色扩展）
严重违反	7 项	0 项（核心实体层已纯化）
已通过修正消除	—	8 项
通过方案简化消除	—	2 项（导出、label_config）

八、AI 原始类图（实验留存）

以下为 P3 实验中 AI 生成的初版类图结构，存在上述 SOLID 违规，**未按此实现**：



AI 原始类图

Mermaid 源文件：P3-AI原始类图.mmd

九、实际修正措施对照

#	原则	AI 初稿违规点	实际修正措施	代码位置
1	S	Task 5 大职责聚合	Task 纯 PO；业务拆入 TaskService / TaskAggregateService / TaskStageSyncService	model/po/Task.java, service/
2	S	Annotation 混入业务	AnnotationPo 纯数据；AnnotationService + AnnotationPersistenceService	model/po/AnnotationPo.java
3	S	User 混入认证	SysUser 纯数据；AuthService 负责 login/logout	service/AuthService.java
4	O	导出格式扩展	前端浏览器 ZIP 导出，后端 export() 废弃	TaskService.export()
5	O	文书来源扩展	TaskDocumentFactory switch 工厂	support/TaskDocumentFactory.java
6	O	配置历史	annotation.guide_snapshot JSON	annotation 表
7	L	Label 语义双重	label_l1 / label_l2 独立表与 PO	model/po/LabelL1.java

8	D	标注 JSON 依赖	proposition / argument_relation / relation_member 规范化	deploy/init.sql
9	D	布局 JSON 依赖	GraphLayoutJsonCodec 隔离编解码	support/GraphLayoutJsonCodec.java
10	D	Mapper 具体实现	MyBatis Mapper 接口 + Spring 注入	mapper/*.java

十、已知技术债

1. **TaskService 职责偏多**：任务、成员、文书、标注加载集中在一个类，后续可拆为 TaskCommandService / TaskQueryService。
2. **无 Service 接口层**：课程设计中规划的 ITaskLifecycle 等未实现，当前依赖 Spring 具体类注入。
3. **密码明文比对**：演示环境 password_hash 可为明文，生产需 BCrypt（AuthService 第 35 行直接 equals）。
4. **角色 ENUM 硬编码**：扩展第三角色需改表结构与应用代码。